b) Investigación

¿Cómo se dividen los patrones GOF y qué significa cada categoría? Realizar una explicación de cada uno de ellos, con su respectivo diagrama de clases y un ejemplo, destacando cuando es conveniente utilizarlos y en qué contexto.

Los patrones GOF se dividen en 3 categorías:

1. Patrones creacionales

   Se encargan de los mecanismos de creación de objetos. Desacoplan al sistema de cómo se crean sus objetos. Es decir que hacen la separación entre el código cliente (que utiliza para algo un determinado objeto) de la creación del objeto que quiere el código cliente. Son 5 patrones:

   - Singleton

     Permite asegurarnos de que una clase tenga una única instancia (esto es bueno para controlar el acceso a algún recurso compartido), a la vez que proporciona un punto de acceso global a dicha instancia (evitar variables globales, ya sabemos que usarlas está mal).

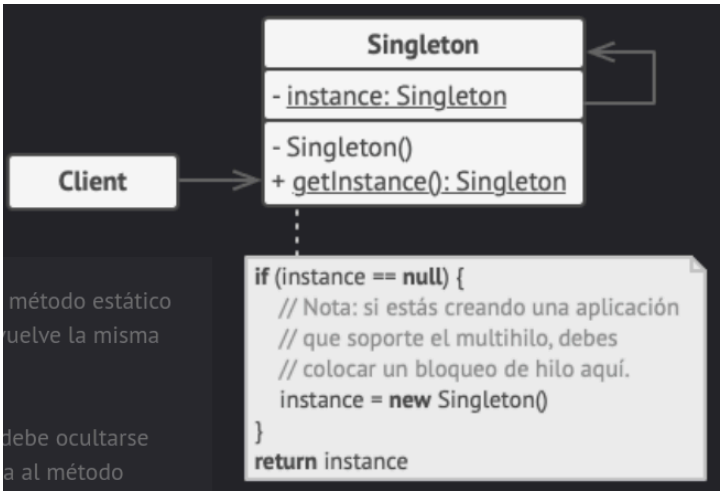
     Todas las implementaciones del patrón Singleton tienen estos dos pasos en común:

     Hacer privado el constructor por defecto para evitar que otros objetos utilicen el operador new con la clase Singleton.

     Crear un método de creación estático que actúe como constructor. Tras bambalinas, este método invoca al constructor privado para crear un objeto y lo guarda en un campo estático. Las siguientes llamadas a este método devuelven el objeto almacenado en caché.

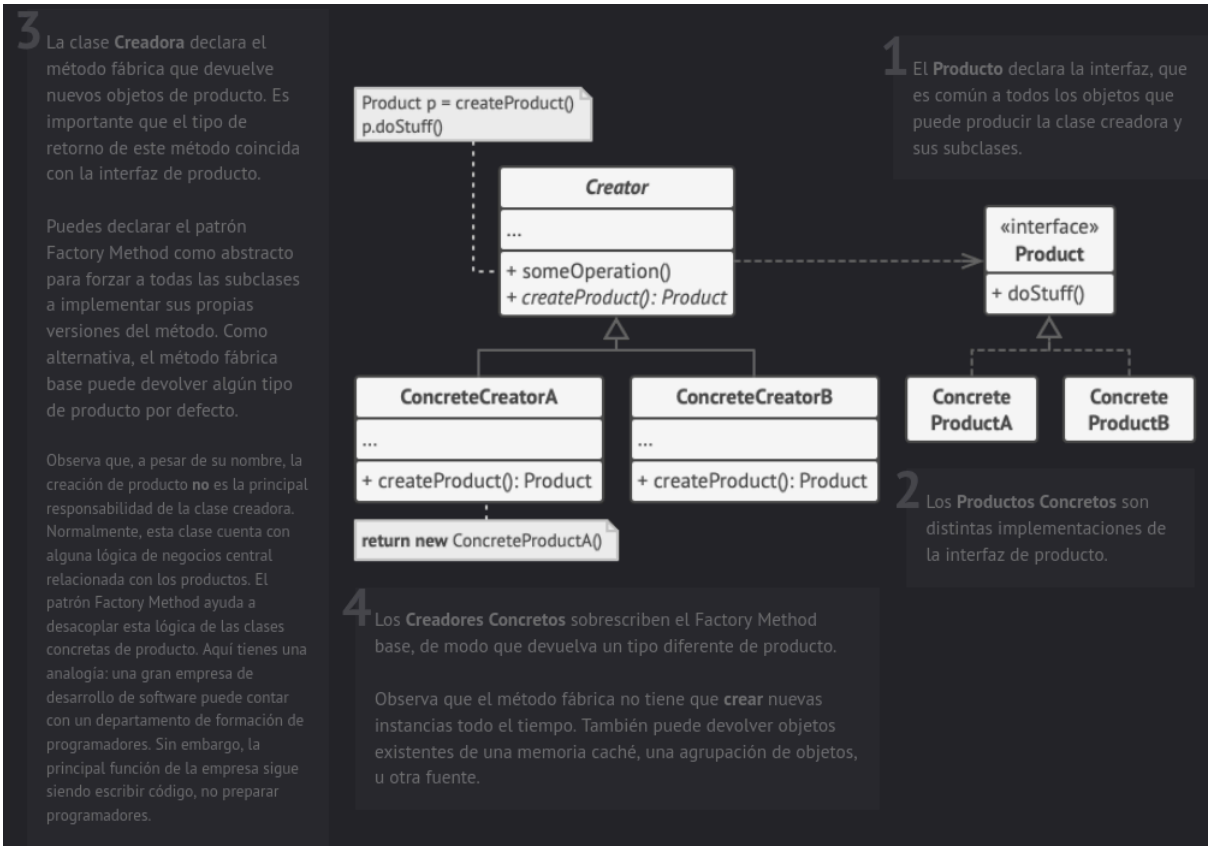
     Si tu código tiene acceso a la clase Singleton, podrá invocar su método estático. De esta manera, cada vez que se invoque este método, siempre se devolverá el mismo objeto.

     EJEMPLO: el gobierno es un ejemplo excelente del patrón Singleton. Un país sólo puede tener un gobierno oficial. Independientemente de las identidades personales de los individuos que forman el gobierno, el título “Gobierno de X” es un punto de acceso global que identifica al grupo de personas a cargo.



- Factory Method

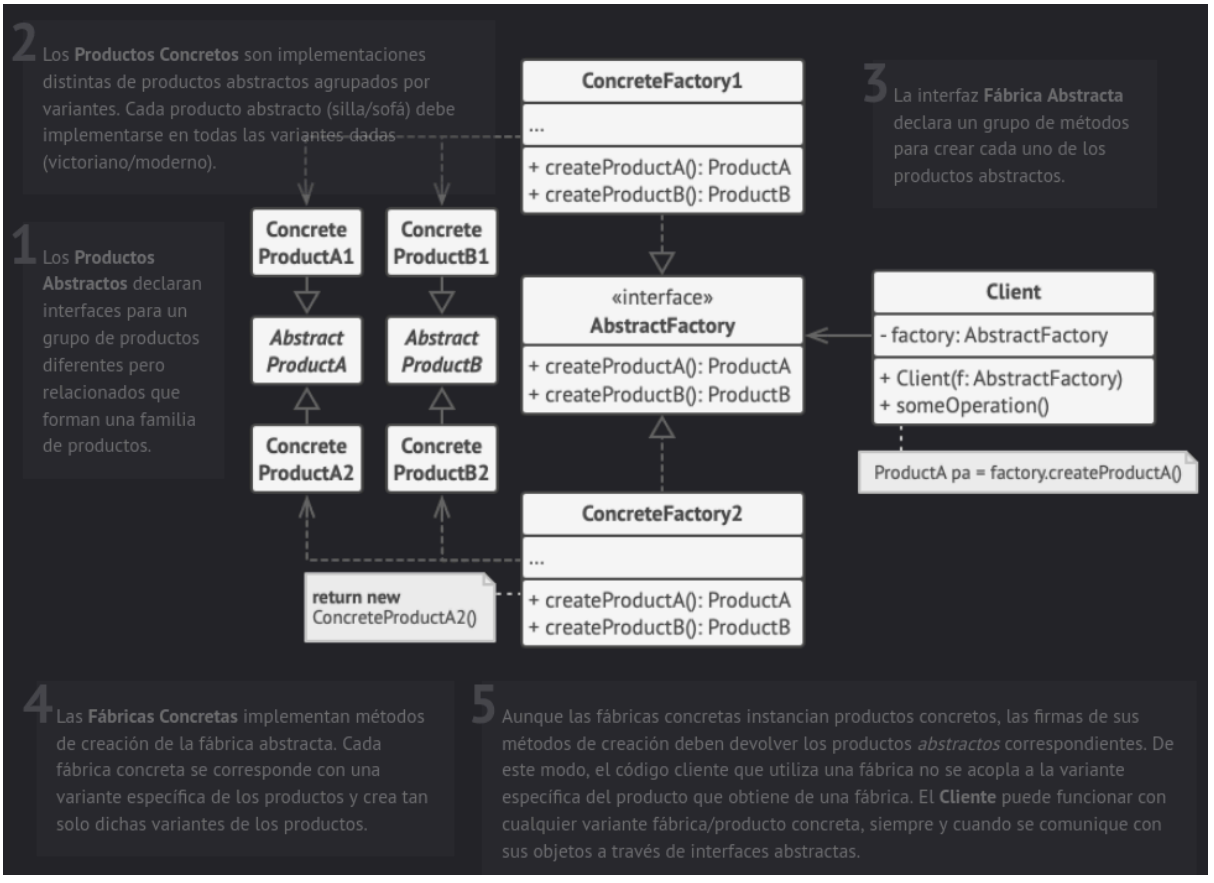
Proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclases alterar el tipo de objetos que se crearán.
El patrón Factory Method sugiere que, en lugar de llamar al operador new para construir objetos directamente, se invoque a un método fábrica especial. Los objetos se siguen creando a través del operador new, pero se invocan desde el método fábrica. Hay una limitación: las subclases sólo pueden devolver productos de distintos tipos si dichos productos tienen una clase base o interfaz común. Además, el método fábrica en la clase base debe tener su tipo de retorno declarado como dicha interfaz.



Ejemplo: tanto la clase Camión como la clase Barco deben implementar la interfaz Transporte, que declara un método llamado entrega. Cada clase implementa este método de forma diferente: los camiones entregan su carga por tierra, mientras que los barcos lo hacen por mar. El método fábrica dentro de la clase LogísticaTerrestre devuelve objetos de tipo camión, mientras que el método fábrica de la clase LogísticaMarítima devuelve barcos.

- Abstract Factory

Permite producir familias de objetos relacionados sin especificar sus clases concretas.

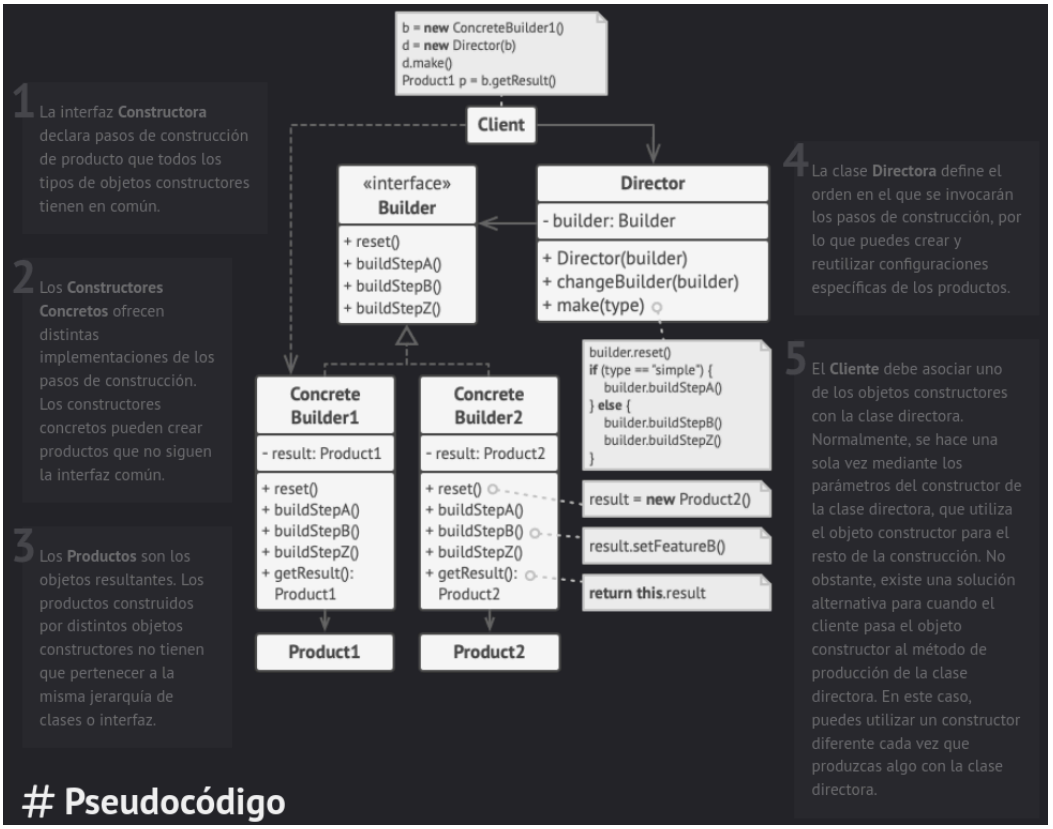


Ejemplo: en una tienda de muebles donde el código está compuesto por clases que representan una familia de productos relacionados, digamos: Silla + Sofá + Mesilla y algunas variantes de esta familia. Por ejemplo, los productos Silla + Sofá + Mesilla están disponibles en estas variantes: Moderna, Victoriana, ArtDecó.
Necesitamos una forma de crear objetos individuales de mobiliario para que combinen con otros objetos de la misma familia.
Declaremos de forma explícita interfaces para cada producto diferente de la familia de productos (por ejemplo, silla, sofá o mesilla). Después podemos hacer que todas las variantes de los productos sigan esas interfaces. El siguiente paso consiste en declarar la Fábrica abstracta: una interfaz con una lista de métodos de creación para todos los productos que son parte de la familia de productos (por ejemplo, crearSilla,

crearSofá y crearMesilla). Estos métodos deben devolver productos abstractos representados por las interfaces que extrajimos previamente: Silla, Sofá, Mesilla, etc. Para cada variante de una familia de productos, creamos una clase de fábrica independiente basada en la interfaz FábricaAbstracta. Una fábrica es una clase que devuelve productos de un tipo particular. Por ejemplo, la FábricadeMueblesModernos sólo puede crear objetos de SillaModerna, SofáModerno y MesillaModerna. Digamos que el cliente quiere una fábrica para producir una silla. El cliente no tiene que conocer la clase de la fábrica y tampoco importa el tipo de silla que obtiene. Ya sea un modelo moderno o una silla de estilo victoriano, el cliente debe tratar a todas las sillas del mismo modo, utilizando la interfaz abstracta Silla. Con este sistema, lo único que sabe el cliente sobre la silla es que implementa de algún modo el método sentarse. Además, sea cual sea la variante de silla devuelta, siempre combinará con el tipo de sofá o mesilla producida por el mismo objeto de fábrica.

- Builder

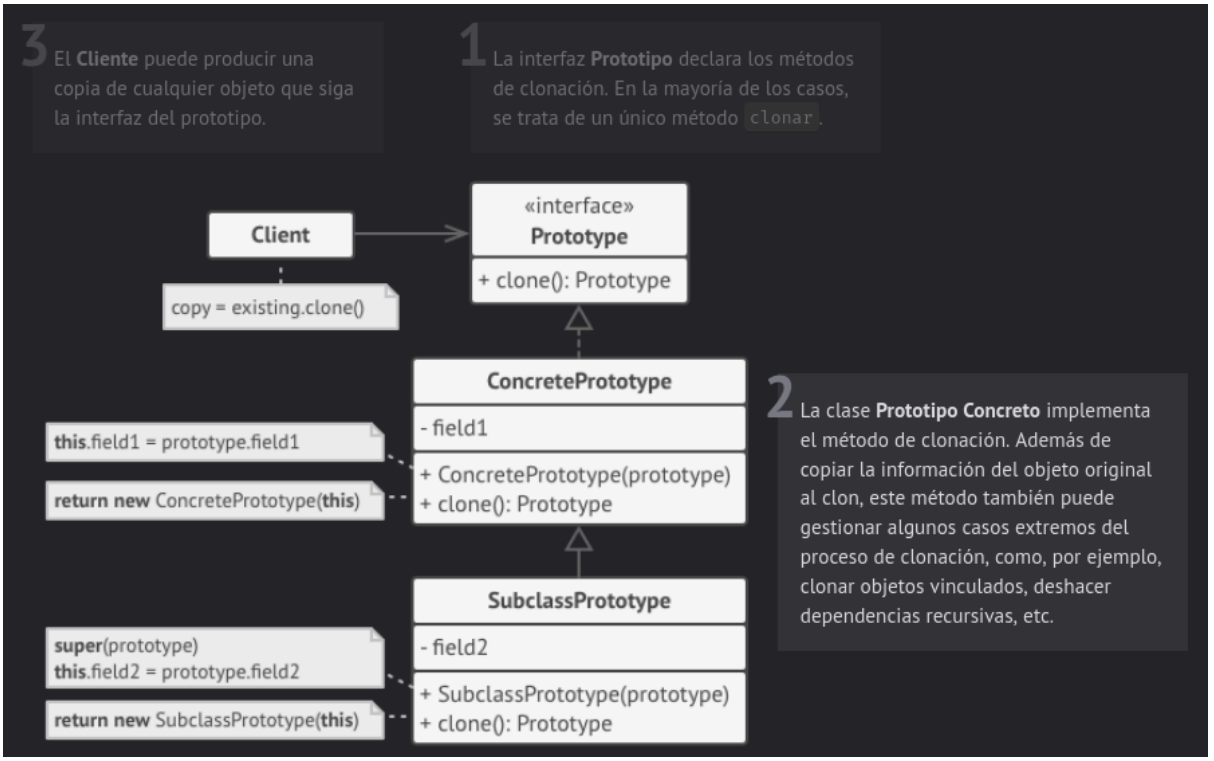
Builder es un patrón de diseño creacional que nos permite construir objetos complejos paso a paso. El patrón nos permite producir distintos tipos y representaciones de un objeto empleando el mismo código de construcción. A diferencia de otros patrones creacionales, no requiere que los productos tengan una interfaz común. Como un objeto complejo que requiere una inicialización laboriosa, paso a paso, de muchos campos y objetos anidados. Normalmente, este código de inicialización está sepultado dentro de un monstruoso constructor con una gran cantidad de parámetros. O, peor aún: disperso por todo el código cliente.
El patrón organiza la construcción de objetos en una serie de pasos. Para crear un objeto, se ejecuta una serie de estos pasos en un objeto constructor. Lo importante es que no necesitas invocar todos los pasos. Puedes invocar sólo aquellos que sean necesarios para producir una configuración particular de un objeto.



Ejemplo: crear un objeto Casa. Para construir una casa sencilla, debemos construir cuatro paredes y un piso, así como instalar una puerta, colocar un par de ventanas y ponerle un tejado. Pero ¿qué pasa si quieres una casa más grande y luminosa, con un jardín y otros extras (como sistema de calefacción, instalación de fontanería y cableado eléctrico)? Para no generar subclases, crear un enorme constructor dentro de la clase base Casa con todos los parámetros posibles para controlar el objeto casa. En la mayoría de los casos, gran parte de los parámetros no se utilizará. El patrón Builder sugiere que saques el código de construcción del objeto de su propia clase y lo coloques dentro de objetos independientes llamados *constructores*.

- Prototype

Permite copiar objetos existentes sin que el código dependa de sus clases. El patrón Prototype delega el proceso de clonación a los propios objetos que están siendo clonados. El patrón declara una interfaz común para todos los objetos que soportan la clonación. Esta interfaz nos permite clonar un objeto sin acoplar el código a la clase de ese objeto. Normalmente, dicha interfaz contiene un único método clonar. Conveniente cuando se quiere evitar crear un objeto complejo desde cero porque su estado inicial varía muy poco respecto a uno ya existente.



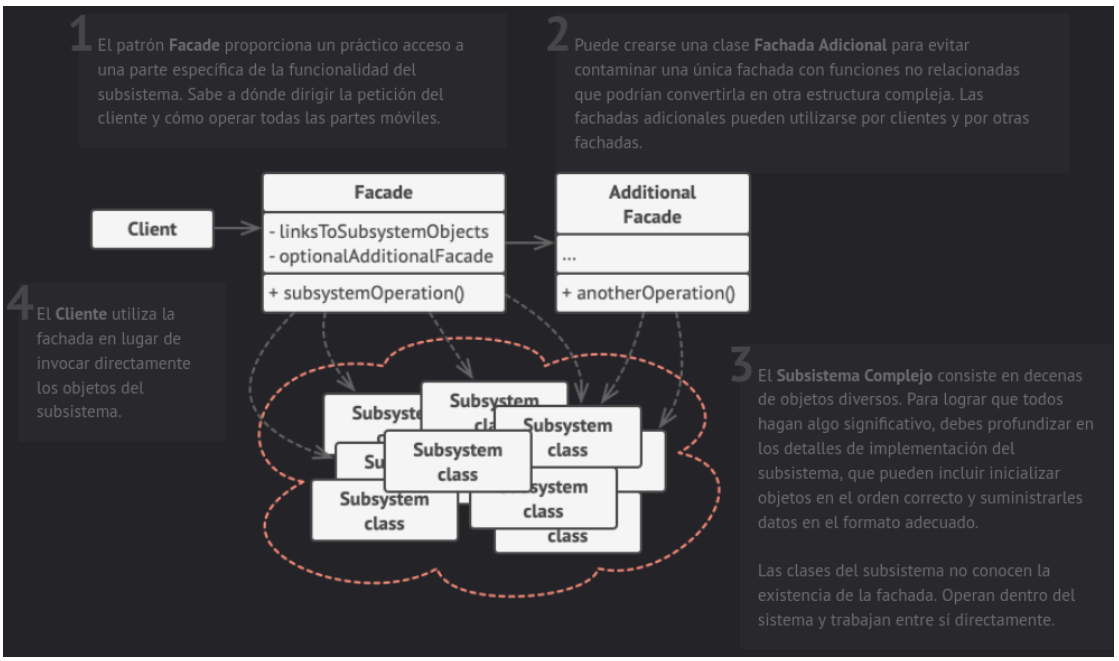
Ejemplo: por ejemplo podemos tener una interfaz “Figura” la cual implementen dos interfaces concretas “Rectangulo” y “Circulo”, y ambas sobre escribirán el método clonar para que se puedan clonar objetos rectangulo y circulo.

2. Patrones Estructurales

Explican cómo ensamblar objetos y clases en estructuras más grandes

- Facade

  Proporciona una interfaz simplificada a una biblioteca, un framework o cualquier otro grupo complejo de clases.
  Una fachada es una clase que proporciona una interfaz simple a un subsistema complejo que contiene muchas partes móviles. Una fachada puede proporcionar una funcionalidad limitada en comparación con trabajar directamente con el subsistema. Sin embargo, tan solo incluye las funciones realmente importantes para los clientes.
  Tener una fachada resulta útil cuando tienes que integrar tu aplicación con una biblioteca sofisticada con decenas de funciones, de la cual sólo necesitas una pequeña parte.

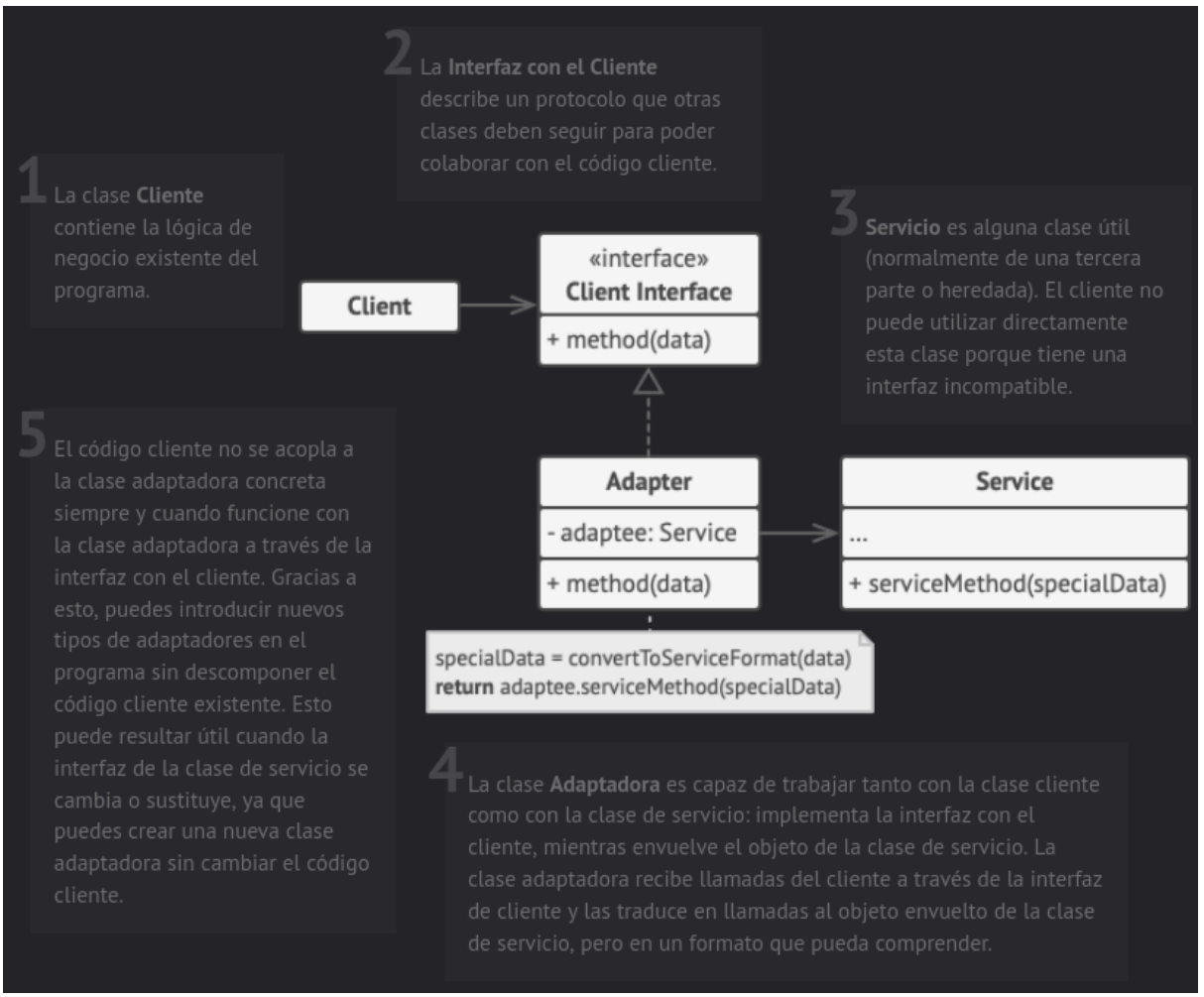


Ejemplo: una aplicación que sube vídeos divertidos de gatos a las redes sociales, podría potencialmente utilizar una biblioteca de conversión de vídeo profesional. Sin embargo, lo único que necesita en realidad es una clase con el método simple codificar(nombreDelArchivo, formato). Una vez que crees dicha clase y la conectes con la biblioteca de conversión de vídeo, tendrás tu primera fachada.

- Adapter

Permite la colaboración entre objetos con interfaces incompatibles. Un adaptador envuelve uno de los objetos para esconder la complejidad de la conversión que tiene lugar tras bambalinas. El objeto envuelto ni siquiera es consciente de la existencia del adaptador. Por ejemplo, puedes envolver un objeto que opera con metros y kilómetros con un adaptador que convierte todos los datos al sistema anglosajón, es decir, pies y millas.

Los adaptadores no solo convierten datos a varios formatos, sino que también ayudan a objetos con distintas interfaces a colaborar. Funciona así: El adaptador obtiene una interfaz compatible con uno de los objetos existentes. Utilizando esta interfaz, el objeto existente puede invocar con seguridad los métodos del adaptador. Al recibir una llamada, el adaptador pasa la solicitud al segundo objeto, pero en un formato y orden que ese segundo objeto espera.

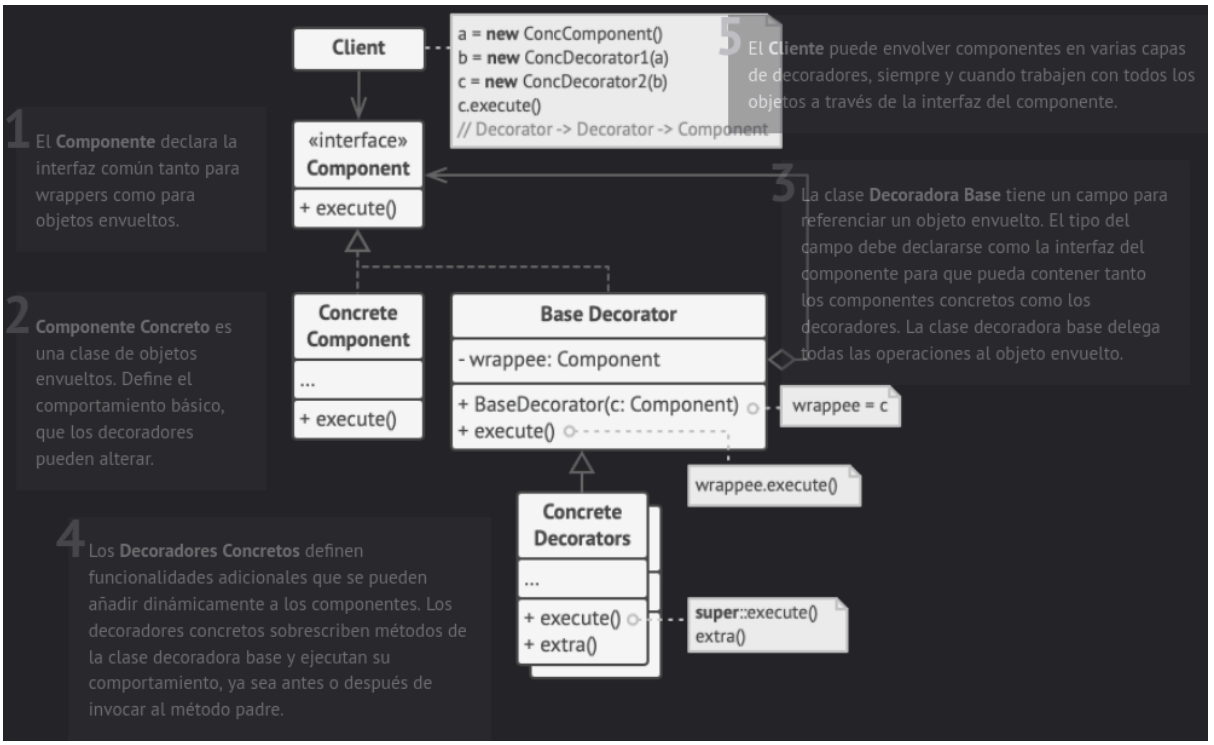


Ejemplo: una aplicación de análisis de datos que obtiene los datos de determinada fuente con formato XML. En cierto momento, decides mejorar la aplicación integrando una inteligente biblioteca de análisis de una tercera persona. Pero hay una trampa: la biblioteca de análisis solo funciona con datos en formato JSON. Puedes crear adaptadores de XML a JSON para cada clase de la biblioteca de análisis con la que trabaje tu código directamente. Después ajustas tu código para que se comunique con la biblioteca únicamente a través de estos adaptadores. Cuando un adaptador recibe una llamada, traduce los datos XML entrantes a una estructura JSON y pasa la llamada a los métodos adecuados de un objeto de análisis envuelto.

- Decorator

permite añadir funcionalidades a objetos colocando estos objetos dentro de objetos encapsuladores especiales que contienen estas funcionalidades.
Cuando tenemos que alterar la funcionalidad de un objeto, lo primero que se viene a la mente es extender una clase. No obstante, la herencia tiene varias limitaciones importantes: es estática y una subclase sólo puede heredar de una clase padre.
El patrón consiste en añadir funcionalidades o responsabilidades nuevas a un objeto dinámicamente envolviéndolo en otro objeto.

En lugar de modificar la clase original o crear una gran cantidad de subclases mediante herencia estática para combinar distintos comportamientos, el patrón Decorator utiliza la composición: hay un objeto original (el componente base). Se coloca dentro de un objeto "envoltorio" (wrapper o decorador). Este objeto envoltorio tiene la misma interfaz que el objeto original, por lo que para el cliente es transparente: le delega el trabajo al objeto envuelto, pero le suma su propio comportamiento extra antes o después de la llamada.
Como todos comparten la misma estructura, puedes envolver un objeto dentro de varios decoradores a la vez (como capas de una cebolla), sumando responsabilidades en tiempo de ejecución según lo necesites.

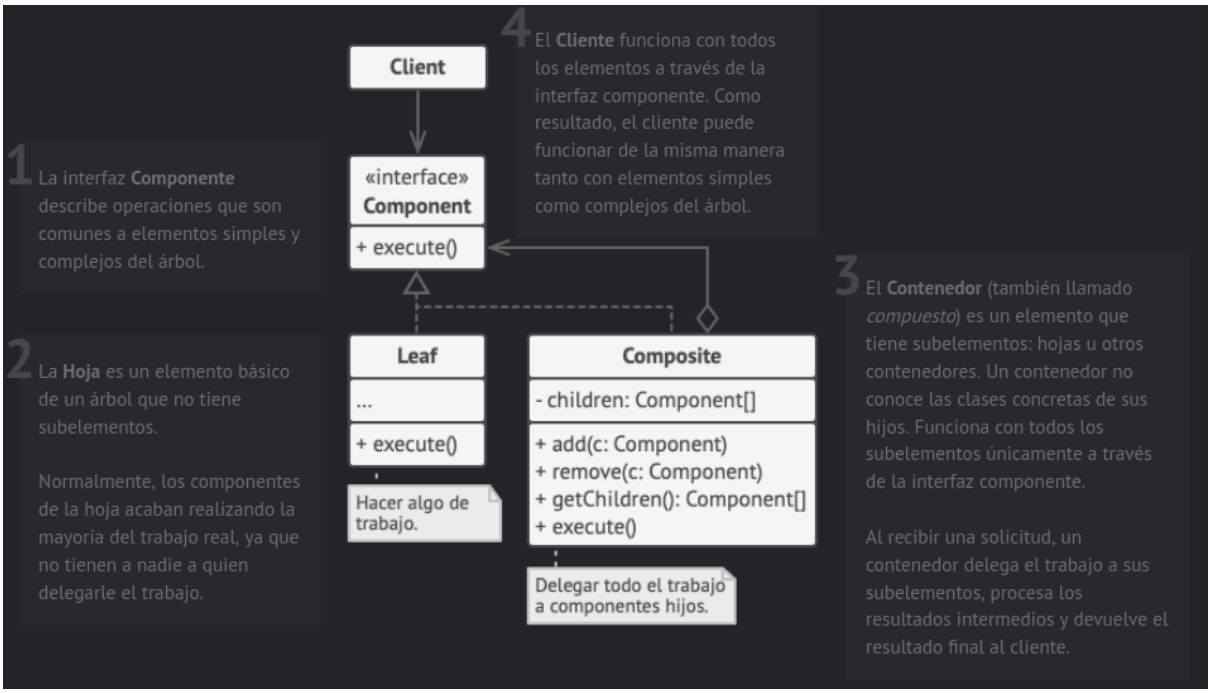


Ejemplo: biblioteca de notificaciones que permite a otros programas notificar a sus usuarios acerca de eventos importantes. La versión inicial de la biblioteca se basaba en la clase Notificador que solo contaba con unos cuantos campos, un constructor y un único método send. El método podía aceptar un argumento de mensaje de un cliente y enviar el mensaje a una lista de correos electrónicos que se pasaban a la clase notificadora a través de su constructor. Una aplicación de un tercero que actuaba como cliente debía crear y configurar el objeto notificador una vez y después utilizarlo cada vez que sucediera algo importante. Si quisiéramos extenderlo para notificaciones por SMS o whatsapp, hacer subclases de notificador no es una buena idea, si un cliente quisiera mandar notificaciones de una combinación de las subclases, habría que hacer más subclases con dichas combinaciones. Dejemos la funcionalidad de las notificaciones por correo electrónico dentro de la clase base Notificador, pero convirtamos el resto de los métodos de notificación en decoradores. El código cliente debe envolver un objeto notificador básico dentro de un grupo de decoradores que satisfagan las preferencias del cliente. Los objetos resultantes se estructurarán como una pila. El último decorador

de la pila será el objeto con el que el cliente trabaja. Debido a que todos los decoradores implementan la misma interfaz que la notificadora base, al resto del código cliente no le importa si está trabajando con el objeto notificador “puro” o con el decorado.

- Composite

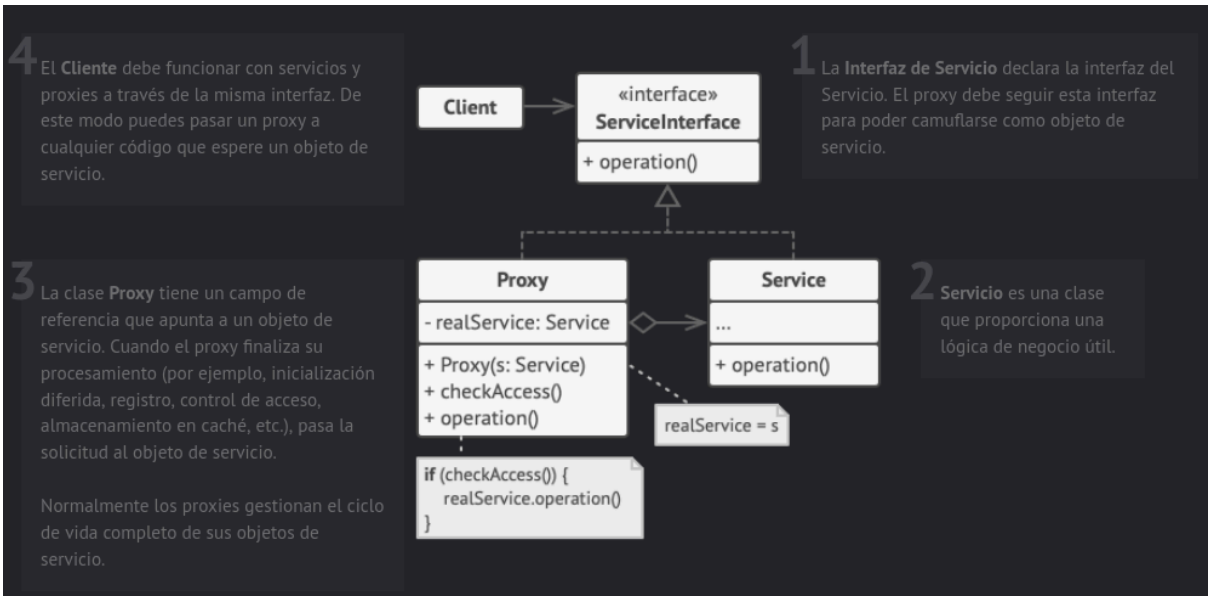
permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales. El uso del patrón Composite sólo tiene sentido cuando el modelo central de tu aplicación puede representarse en forma de árbol.



Ejemplo: un sistema de pedidos donde tienes dos tipos de objetos: Productos y Cajas. Una Caja puede contener varios Productos así como cierto número de Cajas más pequeñas. Estas Cajas pequeñas también pueden contener algunos Productos o incluso Cajas más pequeñas, y así sucesivamente. Los pedidos pueden contener productos sencillos sin envolver, así como cajas llenas de productos... y otras cajas.
¿Cómo determinarás el precio total de ese pedido? El patrón Composite sugiere que trabajes con Productos y Cajas a través de una interfaz común que declara un método para calcular el precio total.
¿Cómo funcionaría este método? Para un producto, sencillamente devuelve el precio del producto. Para una caja, recorre cada artículo que contiene la caja, pregunta su precio y devuelve un total por la caja. Si uno de esos artículos fuera una caja más pequeña, esa caja también comenzaría a repasar su contenido y así sucesivamente, hasta que se calcule el precio de todos los componentes internos. Una caja podría incluso añadir costos adicionales al precio final, como costos de empaquetado.

- Proxy

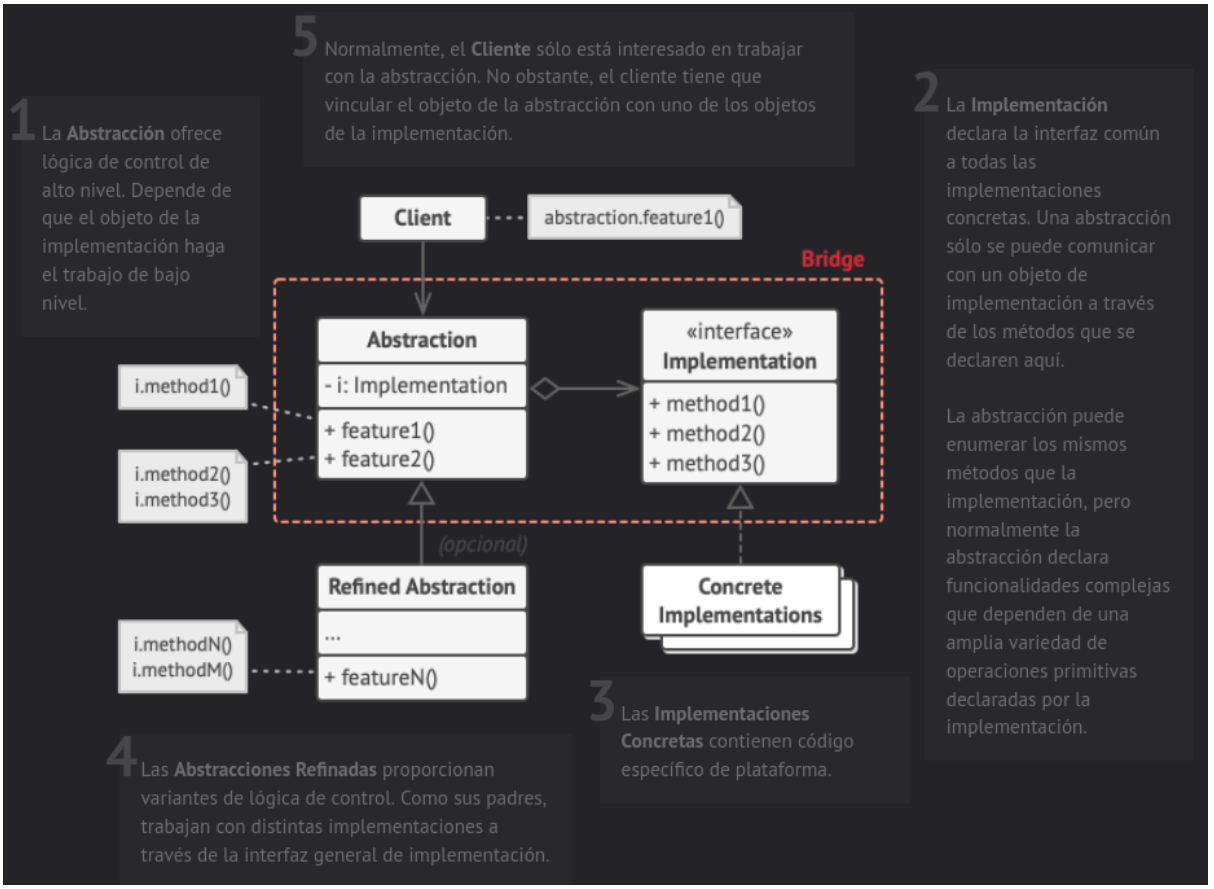
permite proporcionar un sustituto o marcador de posición para otro objeto. Un proxy controla el acceso al objeto original, permitiéndote hacer algo antes o después de que la solicitud llegue al objeto original. El patrón Proxy sugiere que crees una nueva clase proxy con la misma interfaz que un objeto de servicio original. Después actualizas tu aplicación para que pase el objeto proxy a todos los clientes del objeto original. Al recibir una solicitud de un cliente, el proxy crea un objeto de servicio real y le delega todo el trabajo.



Ejemplo: un proxy entre un cliente y un servicio de base de datos. Si 5.000 clientes piden al mismo tiempo el mismo dato, hacer 5.000 consultas idénticas colapsaría la base de datos. El Proxy deja pasar únicamente la primera petición hacia la base de datos, guarda el resultado temporalmente en su memoria rápida y, a los otros 4.999 clientes, les responde al instante entregándoles esa copia sin volver a tocar la base de datos.

- Bridge

permite dividir una clase grande, o un grupo de clases estrechamente relacionadas, en dos jerarquías separadas (abstracción e implementación) que pueden desarrollarse independientemente la una de la otra. Cuando intentamos extender las clases de forma en dos dimensiones independientes el número de clases aumenta exponencialmente.
El patrón Bridge intenta resolver este problema pasando de la herencia a la composición del objeto. Esto quiere decir que se extrae una de las dimensiones a una jerarquía de clases separada, de modo que las clases originales referencian un objeto de la nueva jerarquía, en lugar de tener todo su estado y sus funcionalidades dentro de una clase. Generalmente esas dimensiones que se separan son la abstracción y la implementación.



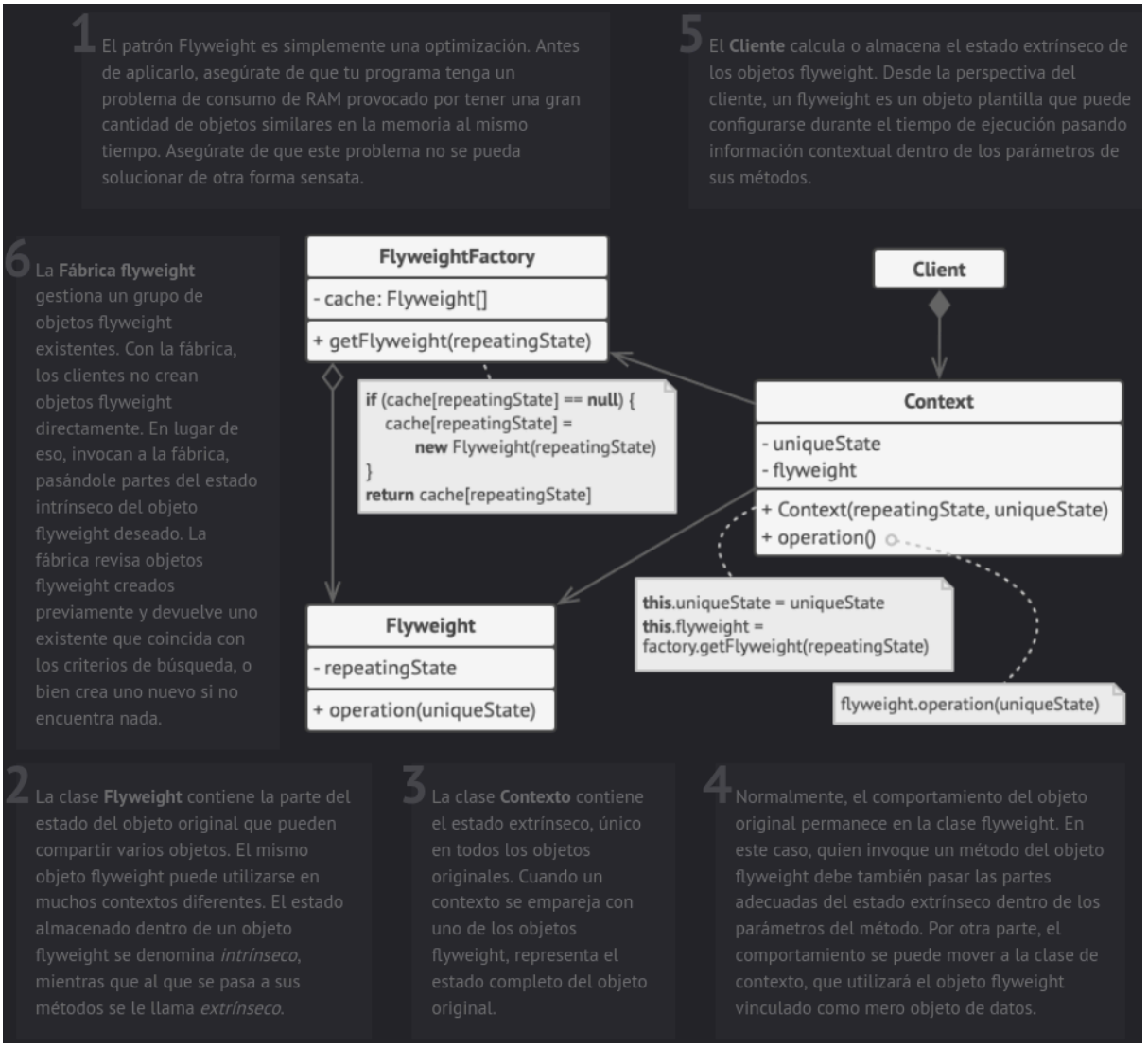
Ejemplo: clase geométrica Forma con un par de subclases: Círculo y Cuadrado. Deseas extender esta jerarquía de clase para que incorpore colores, por lo que planeas crear las subclases de forma Rojo y Azul. Sin embargo, como ya tienes dos subclases, tienes que crear cuatro combinaciones de clase, como CírculoAzul y CuadradoRojo. Con esta solución, podemos extraer el código relacionado con el color y colocarlo dentro de su propia clase, con dos subclases: Rojo y Azul. La clase Forma obtiene entonces un campo de referencia que apunta a uno de los objetos de color. Ahora la forma puede delegar cualquier trabajo relacionado con el color al objeto de color vinculado. Esa referencia actuará como un puente entre las clases Forma y Color. En adelante, añadir nuevos colores no exigirá cambiar la jerarquía de forma y viceversa.

- Flyweight

permite mantener más objetos dentro de la cantidad disponible de RAM compartiendo las partes comunes del estado entre varios objetos en lugar de mantener toda la información en cada objeto.
Esta información constante de un objeto suele denominarse su estado intrínseco. Existe dentro del objeto y otros objetos únicamente pueden leerla, no cambiarla. El resto del estado del objeto, a menudo alterado “desde el exterior” por otros objetos, se denomina el estado extrínseco.

El patrón Flyweight sugiere que dejemos de almacenar el estado extrínseco dentro del objeto. En lugar de eso, debes pasar este estado a métodos específicos que

dependen de él. Tan solo el estado intrínseco se mantiene dentro del objeto, permitiendo que lo reutilices en distintos contextos. Como resultado, necesitarás menos de estos objetos, ya que sólo se diferencian en el estado intrínseco, que cuenta con muchas menos variaciones que el extrínseco.



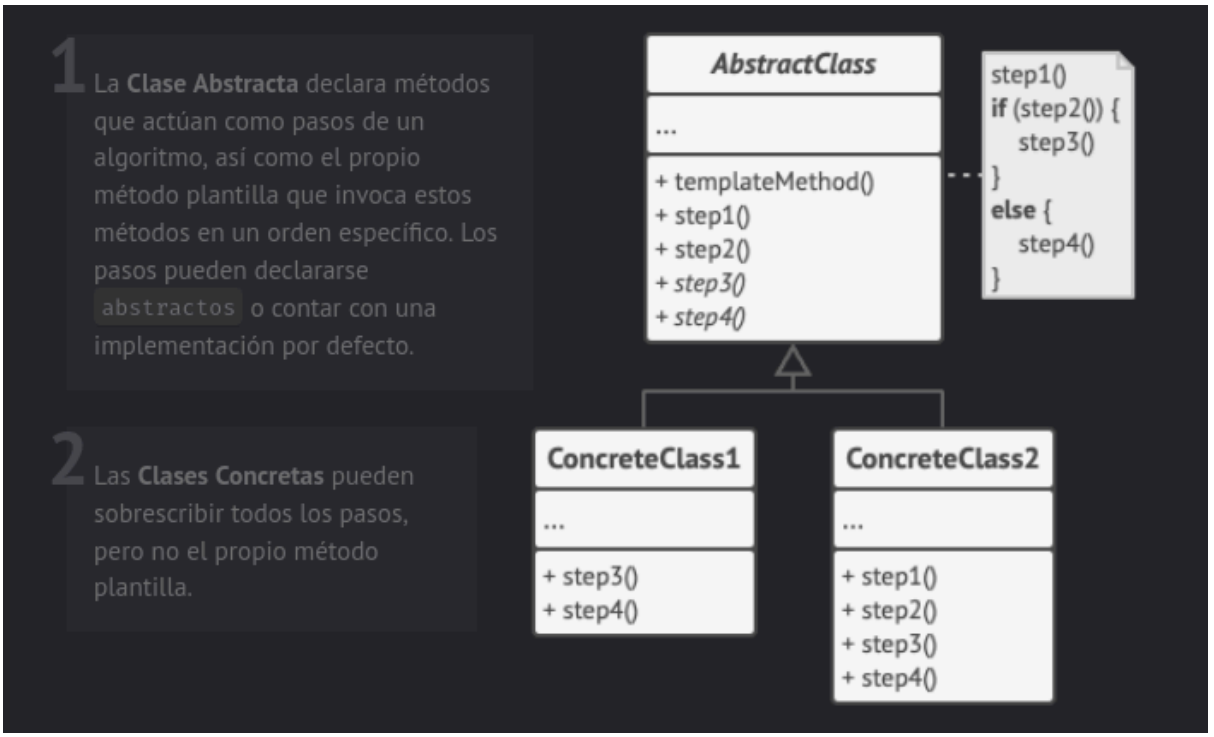
Ejemplo: un videojuego donde hay objetos Bala. Habrá muchos objetos bala instanciados en un momento dado. Las balas tienen datos propios de cada instancia (por ejemplo su posición) pero tiene datos que comunes entre cualquier instancia como color y sprite, que además ocupa mucha RAM. Se crea un solo flyweight con estos atributos que es compartido por todas las balas.

3. Patrones de Comportamiento

Se encargan de los algoritmos, la comunicación efectiva y la asignación de responsabilidades entre objetos.

- Template Method

define el esqueleto de un algoritmo en la superclase pero permite que las subclases sobrescriban pasos del algoritmo sin cambiar su estructura. El patrón Template Method sugiere que dividas un algoritmo en una serie de pasos, conviertas estos pasos en métodos y coloques una serie de llamadas a esos métodos dentro de un único método plantilla. Los pasos pueden ser abstractos, o contar con una implementación por defecto. Para utilizar el algoritmo, el cliente debe aportar su propia subclase, implementar todos los pasos abstractos y sobrescribir algunos de los opcionales si es necesario (pero no el propio método plantilla).



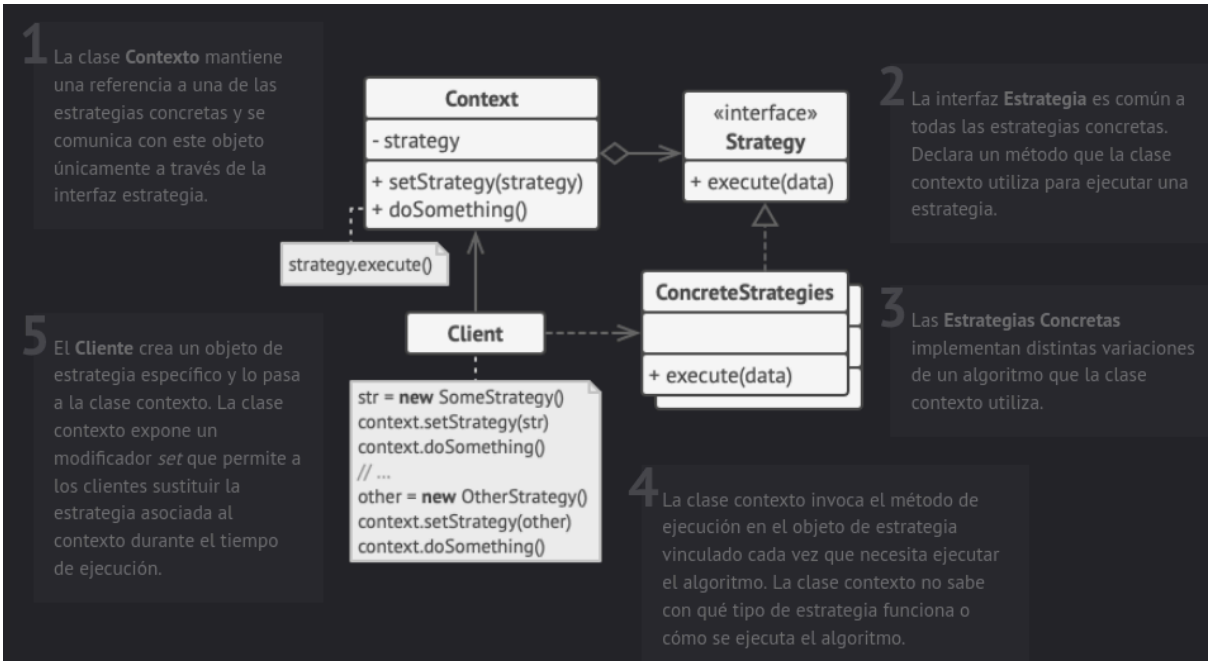
Ejemplo: aplicación de minería de datos. Los usuarios suben a la aplicación documentos en varios formatos (PDF, DOC, CSV) y ésta intenta extraer la información relevante de estos documentos en un formato uniforme.
La primera versión de la aplicación sólo funcionaba con archivos DOC. La siguiente versión podía soportar archivos CSV, después PDF. Parece que el código para abrir/cerrar archivos y extraer/analizar información es diferente para varios formatos de datos, por lo que no tiene sentido tocar estos métodos. No obstante, la implementación de otros pasos, como analizar los datos sin procesar y generar informes, es muy similar, por lo que puede meterse en la clase base, donde las subclases pueden compartir ese código.

- Strategy

permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables. El patrón Strategy sugiere que tomes

esa clase que hace algo específico de muchas formas diferentes y extraigas todos esos algoritmos para colocarlos en clases separadas llamadas *estrategias*.

La clase original, llamada *contexto*, debe tener un campo para almacenar una referencia a una de las estrategias. El contexto delega el trabajo a un objeto de estrategia vinculado en lugar de ejecutarlo por su cuenta. La clase contexto no es responsable de seleccionar un algoritmo adecuado para la tarea. En lugar de eso, el cliente pasa la estrategia deseada a la clase contexto. De hecho, la clase contexto no sabe mucho acerca de las estrategias. Funciona con todas las estrategias a través de la misma interfaz genérica, que sólo expone un único método para disparar el algoritmo encapsulado dentro de la estrategia seleccionada.
De esta forma, el contexto se vuelve independiente de las estrategias concretas, así que puedes añadir nuevos algoritmos o modificar los existentes sin cambiar el código de la clase contexto o de otras estrategias.

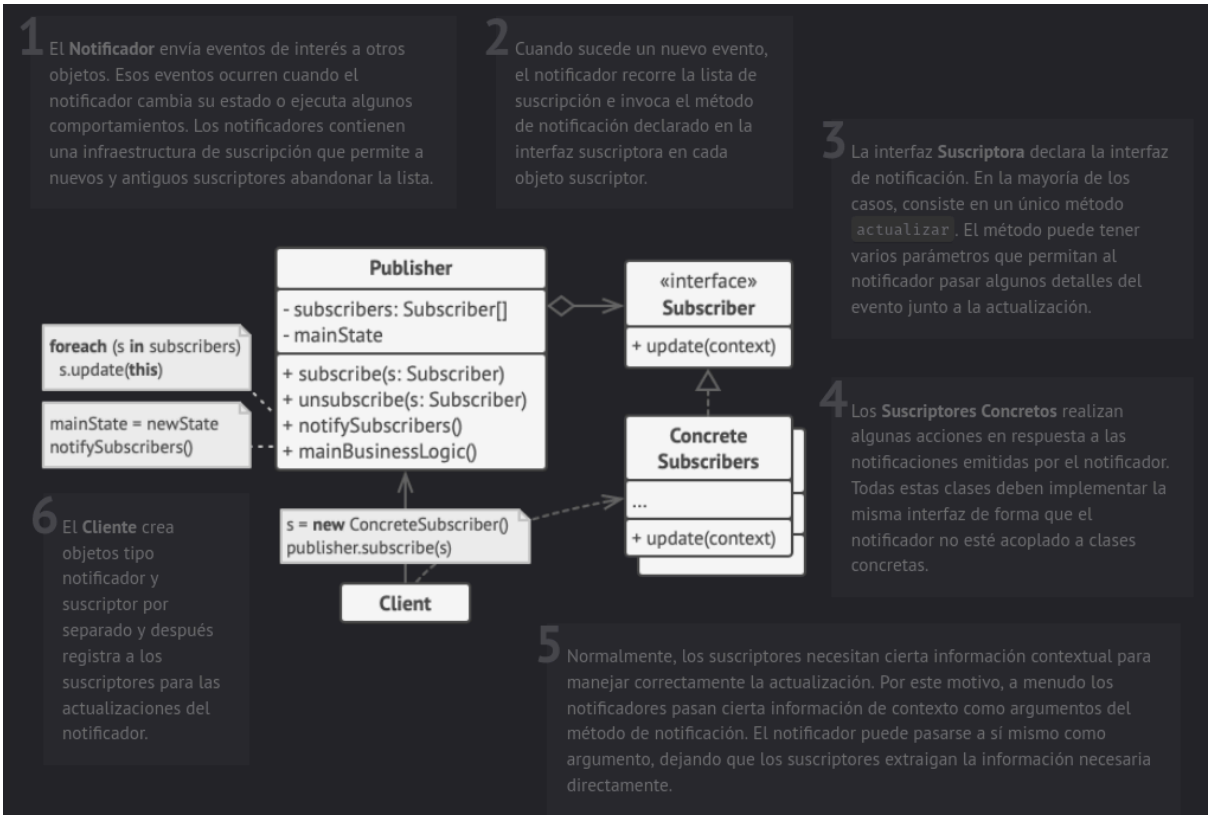


Ejemplo: aplicación de navegación para viajeros ocasionales. Una de las funciones más solicitadas para la aplicación era la planificación automática de rutas. La primera versión de la aplicación sólo podía generar las rutas sobre carreteras. Después se añade esta funcionalidad para viajes a pie, en transporte público, bicicleta, etc. Cada vez que añadías un nuevo algoritmo de enrutamiento, la clase principal del navegador doblaba su tamaño. Cualquier cambio en alguno de los algoritmos, ya fuera un sencillo arreglo de un error o un ligero ajuste de la representación de la calle, afectaba a toda la clase, aumentando las probabilidades de crear un error en un código ya funcional. cada algoritmo de enrutamiento puede extraerse y ponerse en su propia clase con un único método crearRuta. El método acepta un origen y un destino y devuelve una colección de puntos de control de la ruta.

Incluso contando con los mismos argumentos, cada clase de enrutamiento puede crear una ruta diferente. A la clase navegadora principal no le importa qué algoritmo se selecciona ya que su labor principal es representar un grupo de puntos de control en el mapa. La clase tiene un método para cambiar la estrategia activa de enrutamiento, de modo que sus clientes, como los botones en la interfaz de usuario, pueden sustituir el comportamiento seleccionado de enrutamiento por otro.

- Observer

permite definir un mecanismo de suscripción para notificar a varios objetos sobre cualquier evento que le suceda al objeto que están observando.
El objeto que tiene un estado interesante suele denominarse sujeto, pero, como también va a notificar a otros objetos los cambios en su estado, le llamaremos notificador (en ocasiones también llamado publicador). El resto de los objetos que quieren conocer los cambios en el estado del notificador, se denominan suscriptores.
El patrón Observer sugiere que añadas un mecanismo de suscripción a la clase notificadora para que los objetos individuales puedan suscribirse o cancelar su suscripción a un flujo de eventos que proviene de esa notificadora.
Por eso es fundamental que todos los suscriptores implementen la misma interfaz y que el notificador únicamente se comunique con ellos a través de esa interfaz. Esta interfaz debe declarar el método de notificación junto con un grupo de parámetros que el notificador puede utilizar para pasar cierta información contextual con la notificación.

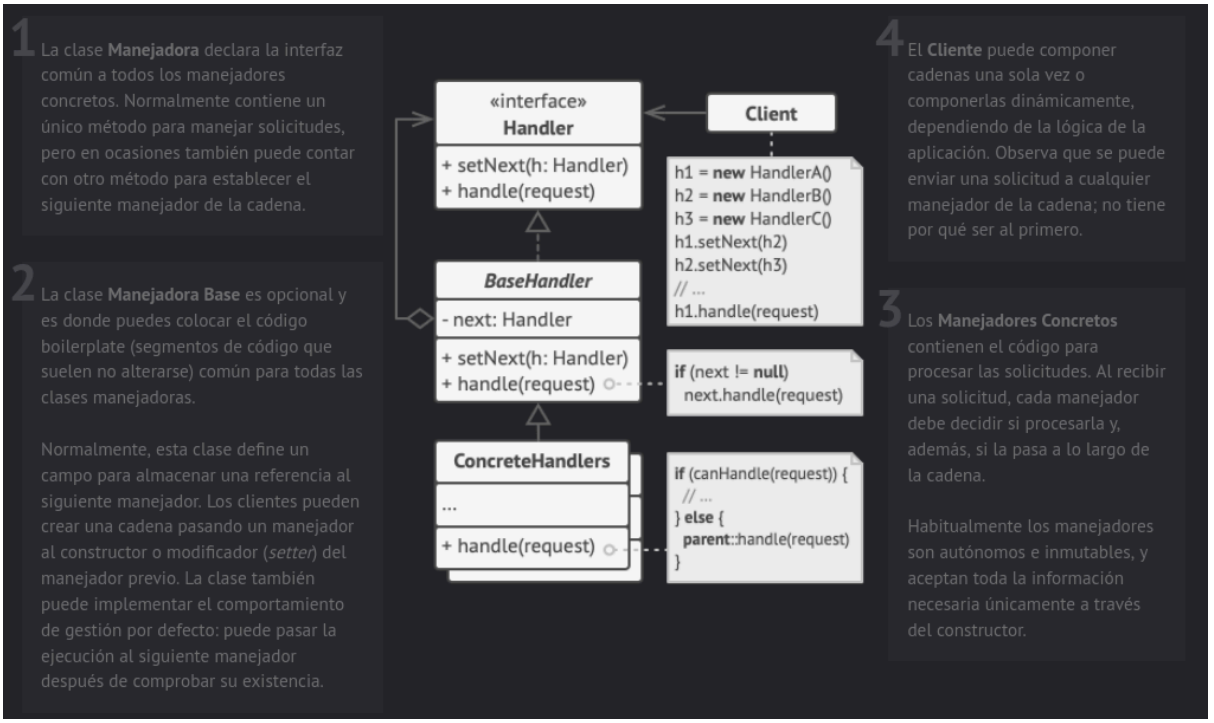


Ejemplo: en una tienda virtual cuando un cliente quiere recibir notificación cuando se reponga el stock de un determinado producto (notificador)

- Chain of Responsibility

permite pasar solicitudes a lo largo de una cadena de manejadores. Al recibir una solicitud, cada manejador decide si la procesa o si la pasa al siguiente manejador de la cadena. Chain of Responsibility se basa en transformar comportamientos particulares en objetos autónomos llamados manejadores. En nuestro caso, cada comprobación debe ponerse dentro de su propia clase con un único método que realice la comprobación. La solicitud, junto con su información, se pasa a este método como argumento.

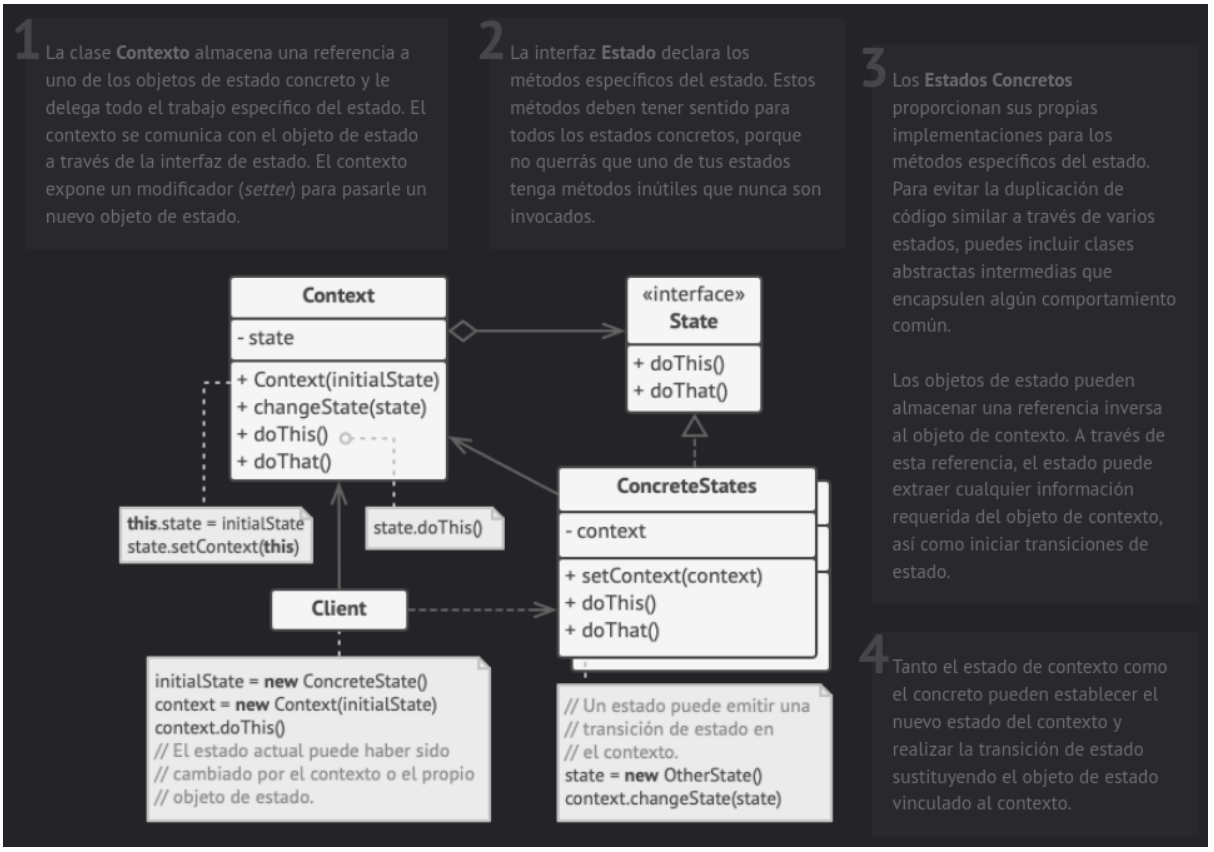
El patrón sugiere que vincules esos manejadores en una cadena. Cada manejador vinculado tiene un campo para almacenar una referencia al siguiente manejador de la cadena.



- State

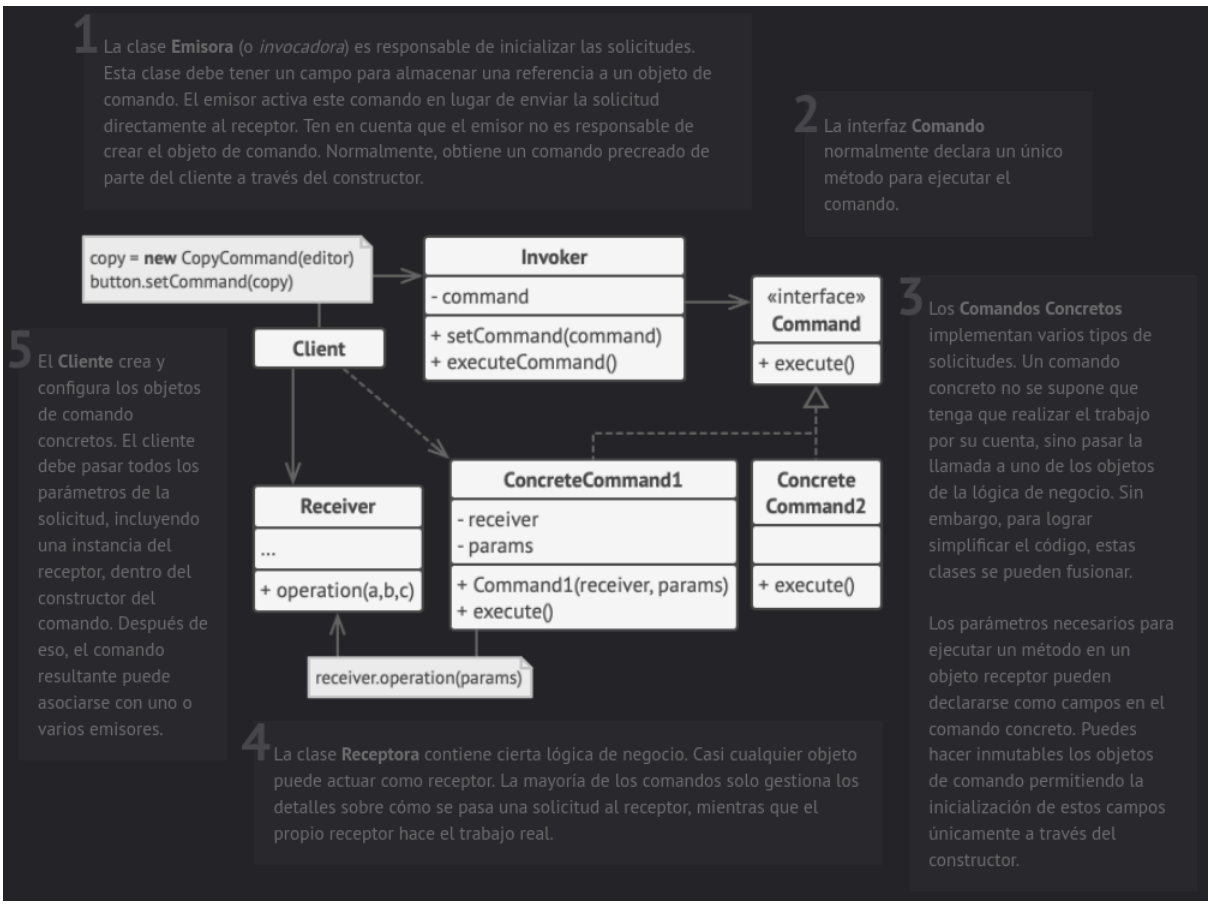
permite a un objeto alterar su comportamiento cuando su estado interno cambia. Parece como si el objeto cambiara su clase. La idea principal es que, en cualquier momento dado, un programa puede encontrarse en un número finito de estados. Dentro de cada estado único, el programa se comporta de forma diferente y puede cambiar de un estado a otro instantáneamente. Sin embargo, dependiendo de un estado actual, el programa puede cambiar o no a otros estados. Estas normas de cambio llamadas transiciones también son finitas y predeterminadas. Una implementación con condicionales es muy sucia.

El patrón State sugiere que crees nuevas clases para todos los estados posibles de un objeto y extraigas todos los comportamientos específicos del estado para colocarlos dentro de esas clases.
En lugar de implementar todos los comportamientos por su cuenta, el objeto original, llamado *contexto*, almacena una referencia a uno de los objetos de estado que representa su estado actual y delega todo el trabajo relacionado con el estado a ese objeto.



- Command

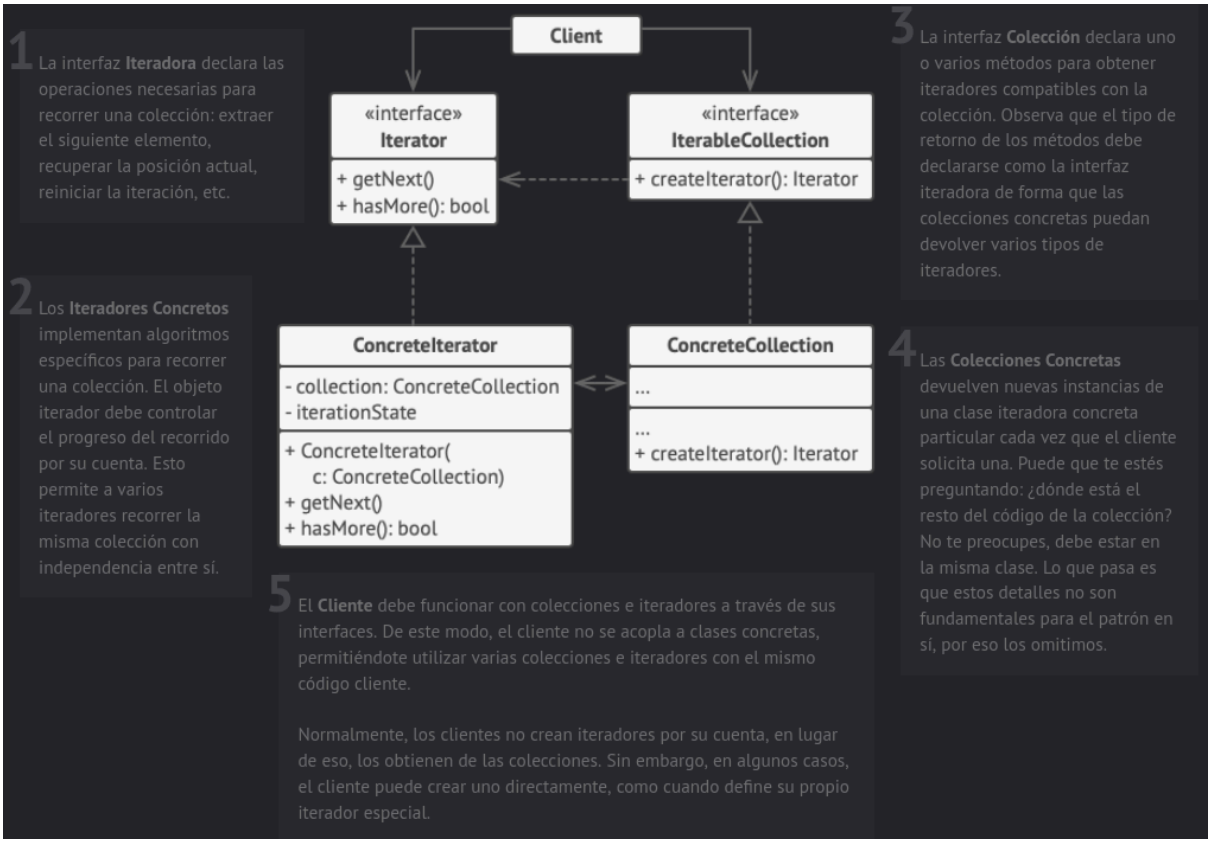
convierte una solicitud en un objeto independiente que contiene toda la información sobre la solicitud. Esta transformación te permite parametrizar los métodos con diferentes solicitudes, retrasar o poner en cola la ejecución de una solicitud y soportar operaciones que no se pueden realizar.



Ejemplo: El ejemplo más habitual es tener una capa para la interfaz gráfica de usuario (GUI) y otra capa para la lógica de negocio. La capa GUI es responsable de representar una bonita imagen en pantalla, capturar entradas y mostrar resultados de lo que el usuario y la aplicación están haciendo. Sin embargo, cuando se trata de hacer algo importante, como calcular la trayectoria de la luna o componer un informe anual, la capa GUI delega el trabajo a la capa subyacente de la lógica de negocio.

- Iterator

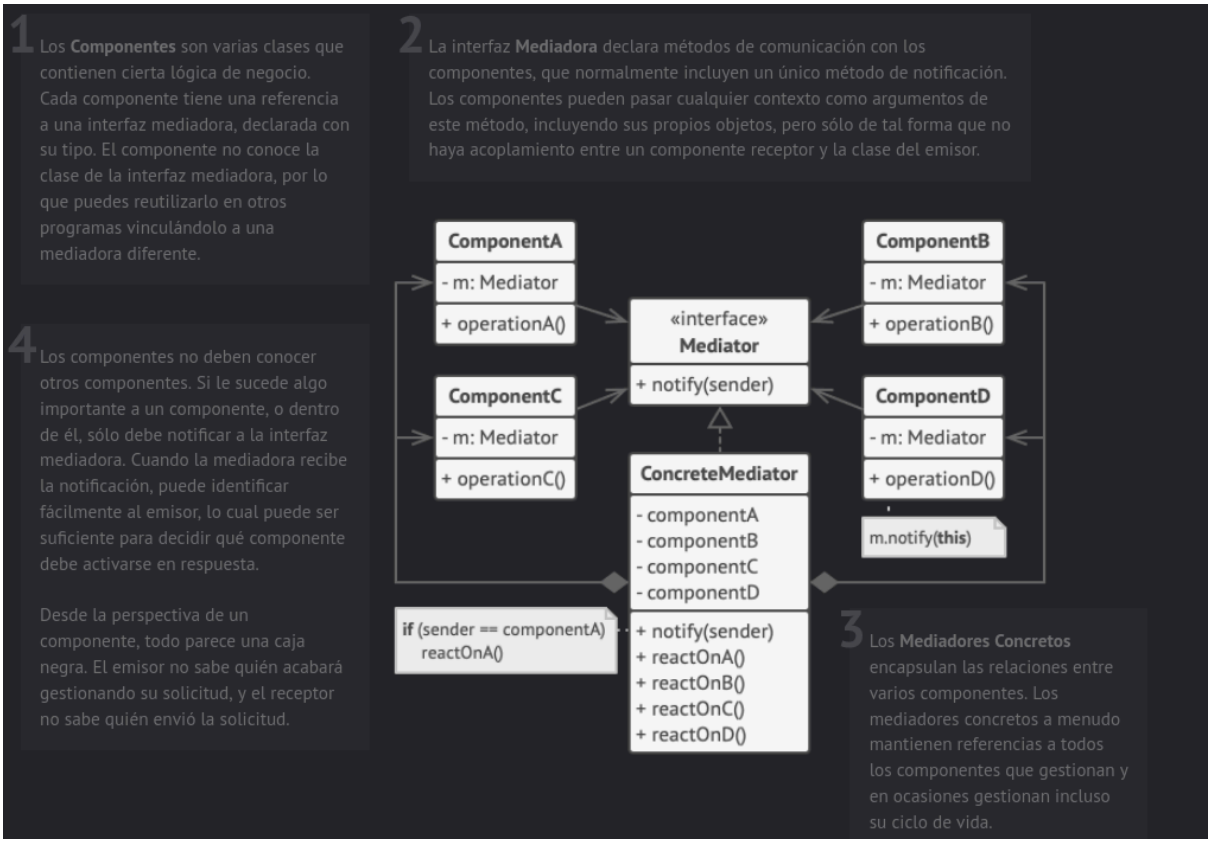
permite recorrer elementos de una colección sin exponer su representación subyacente (lista, pila, árbol, etc.). se basan en pilas, árboles, grafos y otras estructuras complejas de datos.
La idea central del patrón Iterator es extraer el comportamiento de recorrido de una colección y colocarlo en un objeto independiente llamado *iterador*.
Además de implementar el propio algoritmo, un objeto iterador encapsula todos los detalles del recorrido, como la posición actual y cuántos elementos quedan hasta el final. Debido a esto, varios iteradores pueden recorrer la misma colección al mismo tiempo, independientemente los unos de los otros.



Ejemplo: ¿cómo recorres secuencialmente elementos de una estructura compleja de datos, como un árbol? Por ejemplo, un día puede bastarte con un recorrido de profundidad de un árbol, pero, al día siguiente, quizá necesites un recorrido en anchura. Añadir más y más algoritmos de recorrido a la colección nubla gradualmente su responsabilidad principal, que es el almacenamiento eficiente de la información.

- Mediator

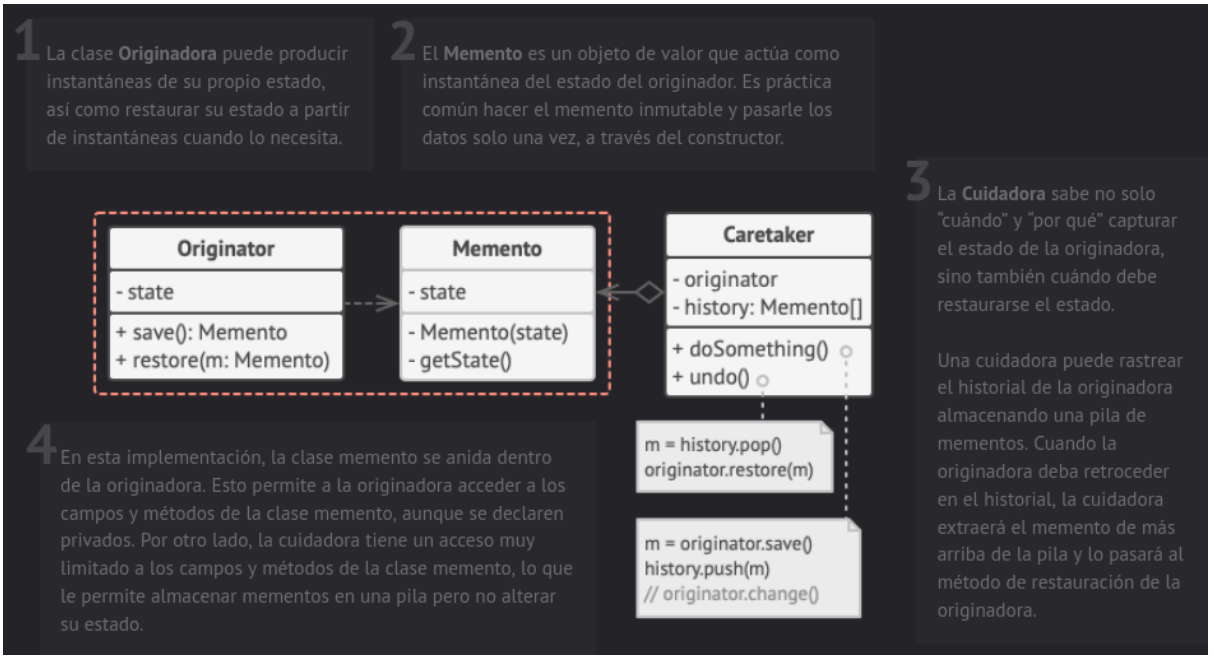
permite reducir las dependencias caóticas entre objetos. El patrón restringe las comunicaciones directas entre los objetos, forzándolos a colaborar únicamente a través de un objeto mediador. El patrón Mediator sugiere que detengas toda comunicación directa entre los componentes que quieres hacer independientes entre sí. En lugar de ello, estos componentes deberán colaborar indirectamente, invocando un objeto mediador especial que redireccione las llamadas a los componentes adecuados. Como resultado, los componentes dependen únicamente de una sola clase mediadora, en lugar de estar acoplados a decenas de sus colegas.



Ejemplo: diálogo para crear y editar perfiles de cliente. Consiste en varios controles de formulario, como campos de texto, casillas, botones, etc. Algunos de los elementos del formulario pueden interactuar con otros. Por ejemplo, al seleccionar la casilla “tengo un perro” puede aparecer un campo de texto oculto para introducir el nombre del perro. Otro ejemplo es el botón de envío que tiene que validar los valores de todos los campos antes de guardar la información. En nuestro ejemplo del formulario de edición de perfiles, la propia clase de diálogo puede actuar como mediadora. Lo más probable es que la clase de diálogo conozca ya todos sus subelementos, por lo que ni siquiera será necesario que introduzcas nuevas dependencias en esta clase.

- Memento

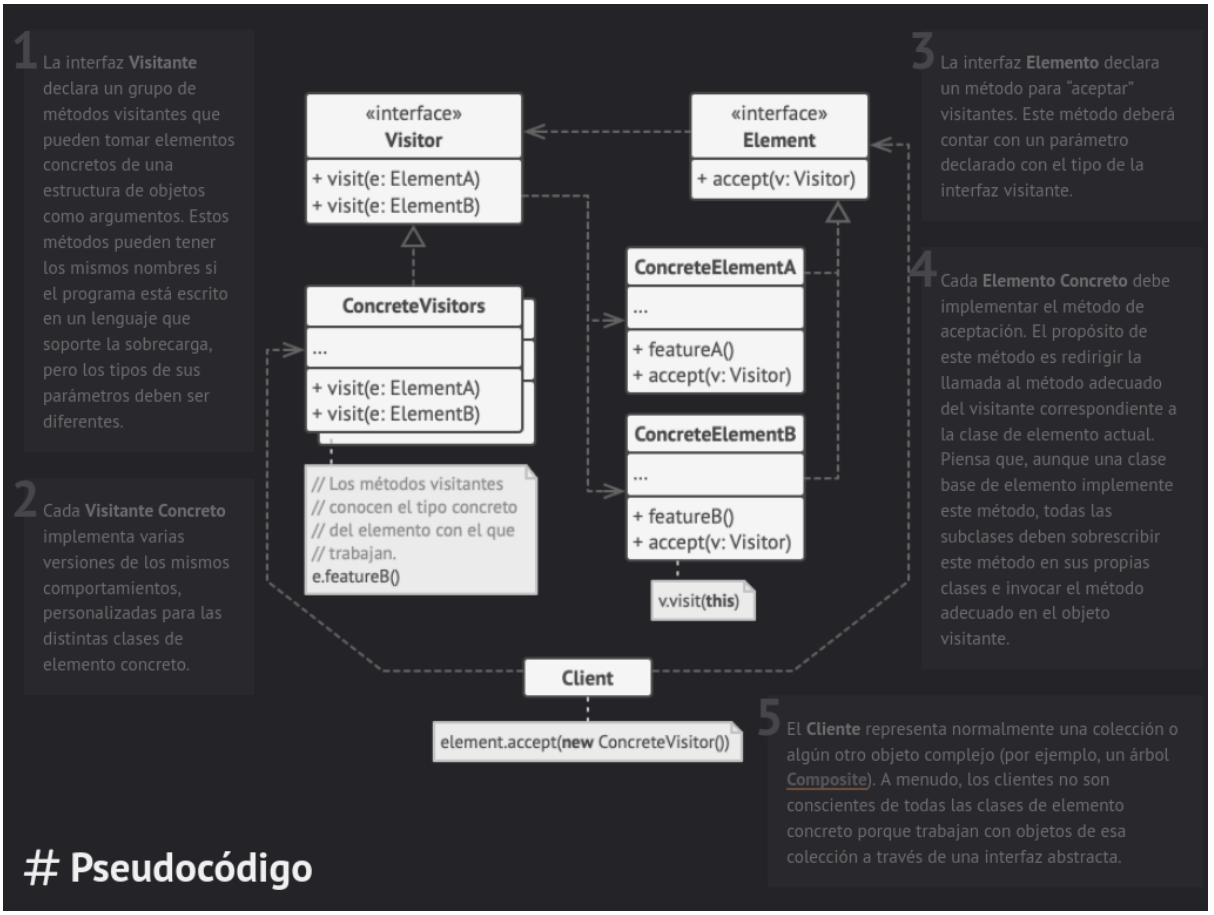
permite guardar y restaurar el estado previo de un objeto sin revelar los detalles de su implementación. El patrón Memento delega la creación de instantáneas de estado al propietario de ese estado, el objeto originador. Por lo tanto, en lugar de que haya otros objetos intentando copiar el estado del editor desde el “exterior”, la propia clase editora puede hacer la instantánea, ya que tiene pleno acceso a su propio estado.
El patrón sugiere almacenar la copia del estado del objeto en un objeto especial llamado memento. Los contenidos del memento no son accesibles para ningún otro objeto excepto el que lo produjo. Otros objetos deben comunicarse con mementos utilizando una interfaz limitada que pueda permitir extraer los metadatos de la instantánea (tiempo de creación, el nombre de la operación realizada, etc.), pero no el estado del objeto original contenido en la instantánea.



Ejemplo: aplicación de edición de texto. Además de editar texto, tu programa puede formatearlo, asi como insertar imágenes en línea, etc.
En cierto momento, decides permitir a los usuarios deshacer cualquier operación realizada en el texto. Antes de realizar cualquier operación, la aplicación registra el estado de todos los objetos y lo guarda en un almacenamiento. Más tarde, cuando un usuario decide revertir una acción, la aplicación extrae la última instantánea del historial y la utiliza para restaurar el estado de todos los objetos. Cuando un usuario activa la función Deshacer, el historial toma el memento más reciente de la pila y lo pasa de vuelta al editor, solicitando una restauración. Debido a que el editor tiene pleno acceso al memento, cambia su propio estado con los valores tomados del memento.

- Visitor

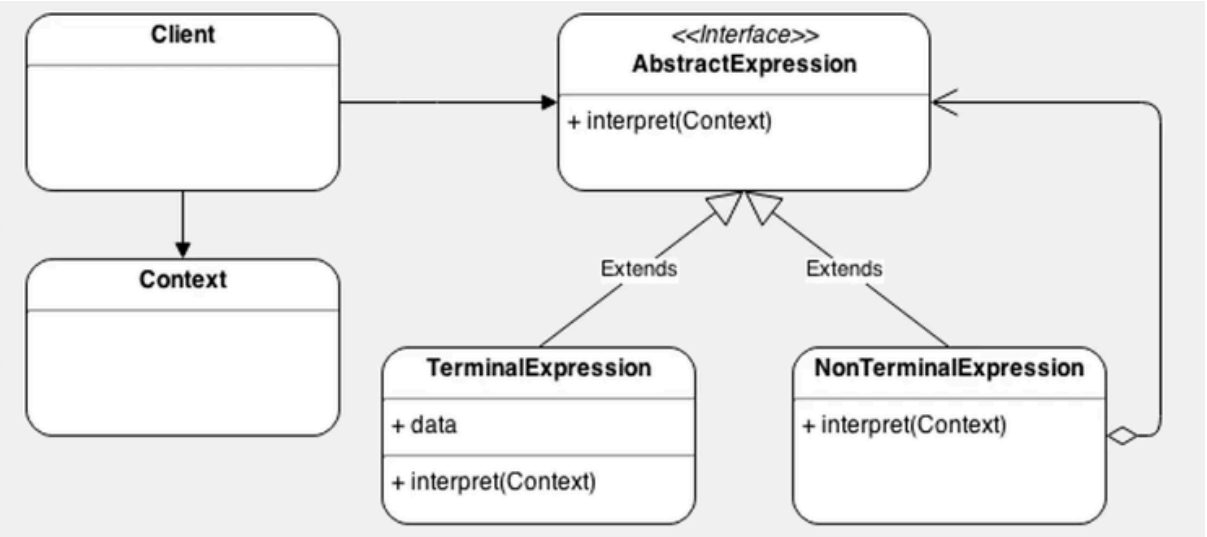
permite separar algoritmos de los objetos sobre los que operan.El patrón Visitor sugiere que coloques el nuevo comportamiento en una clase separada llamada *visitante*, en lugar de intentar integrarlo dentro de clases existentes. El objeto que originalmente tenía que realizar el comportamiento se pasa ahora a uno de los métodos del visitante como argumento, de modo que el método accede a toda la información necesaria contenida dentro del objeto.



Ejemplo: carrito de un supermercado donde tenemos distintos tipos de productos, como alimentos de canasta básica, artículos de electrónica y bebidas alcohólicas. Si necesitamos calcular los impuestos de la compra, en lugar de programar la fórmula impositiva dentro de cada clase de producto y ensuciar su código cada vez que cambia una ley, creamos un objeto externo llamado "CalculadorDeImpuestos" (el visitante) que recorre el carrito visitando cada artículo uno por uno. A medida que pasa por cada producto, este visitante aplica la regla correspondiente según el tipo de objeto (por ejemplo, 0% a los alimentos, 21% a la electrónica y un impuesto extra al alcohol), lo que nos permite agregar nuevas operaciones en el futuro creando un nuevo visitante sin tener que modificar nunca las clases originales de los productos.

- Interpreter

Define una representación para la gramática de un lenguaje junto con un intérprete que utiliza dicha representación para evaluar expresiones del lenguaje. Convierte las reglas de un lenguaje en una estructura de clases. Cada regla o símbolo gramatical se modela como un objeto, y juntas forman un árbol sintáctico abstracto.



Ejemplo: una calculadora que evalúa 2 + 3 * 4:
1. El cliente arma un árbol con nodos terminales (2, 3, 4) y no terminales (+, *).
2. Se recorre el árbol evaluando las prioridades y reglas definidas.
3. El intérprete devuelve el resultado final (14)

TODA LA INFO FUE EXTRAÍDA DE:

https://refactoring.guru/es

https://reactiveprogramming.io/blog/es/patrones-de-diseno/interpreter